

Opción múltiple

1. ¿Qué característica define a una función recursiva?

- a. Siempre retorna *void*
- b. Utiliza bucles *for* o *while*
- c. Se llama a sí misma
- d. Solo se puede usar una vez

2. Lea la siguiente función y seleccione qué es lo que entiende que hace:

```
int funcionMisteriosa(int numero) {  
    if (numero <= 0) {  
        return 0;  
    }  
    return 1 + funcionMisteriosa(numero - 1);  
}
```

- a. Multiplica el número por 1
- b. Cuenta desde el número hasta 0 inclusive
- c. Cuenta desde el 0 hasta el número
- d. Cuenta cuántas veces se puede restar 1 a n hasta 0

3. Lea la siguiente función y seleccione qué es lo que entiende que hace sabiendo que la misma fue invocada como `funcionMisteriosa(3)`:

```
void funcionMisteriosa(int numero) {  
    if (numero == 0) {  
        return;  
    }  
    funcionMisteriosa(numero - 1);  
    System.out.print(n);  
}
```

- a. 123
- b. 321
- c. 111
- d. 333
- e. Error

Completar

1. Completa la función para calcular el doble de un número de forma recursiva:

```
int calcularDoble(int numero) {  
    if (numero == 0) {  
        return 0;  
    }  
    return 2 + calcularDoble(numero - 1);  
}
```

2. Completa la función para calcular la suma de los dígitos de un número de forma recursiva:

```
int sumarDigitos(int numero) {  
    if (numero == 0) {  
        return 0;  
    }  
    return (numero % 10) + sumarDigitos(numero / 10);  
}
```

3. Completa la función para calcular la longitud de un String de forma recursiva:

```
int calcularLongitud(String frase) {  
    if (frase.isEmpty()) {  
        return 0;  
    }  
    return 1 + calcularLongitud(frase.substring(1));  
}
```

Ayuda: recordemos que `String.substring(inicio)` permite obtener el substring desde el punto dado.

4. Un error común en funciones recursivas es olvidar la caso base, lo que puede provocar una excepción de tipo stack overflow en Java.

Desarrollar

1. Escribir un método que calcule la potencia de un número utilizando recursividad.
2. Encontrar y explicar el error en el siguiente código recursivo si lo hubiera:

```
int suma(int n) {  
    return n + suma(n - 1);  
}
```

3. Implementa una función recursiva que cuente cuántos dígitos tiene un número positivo.

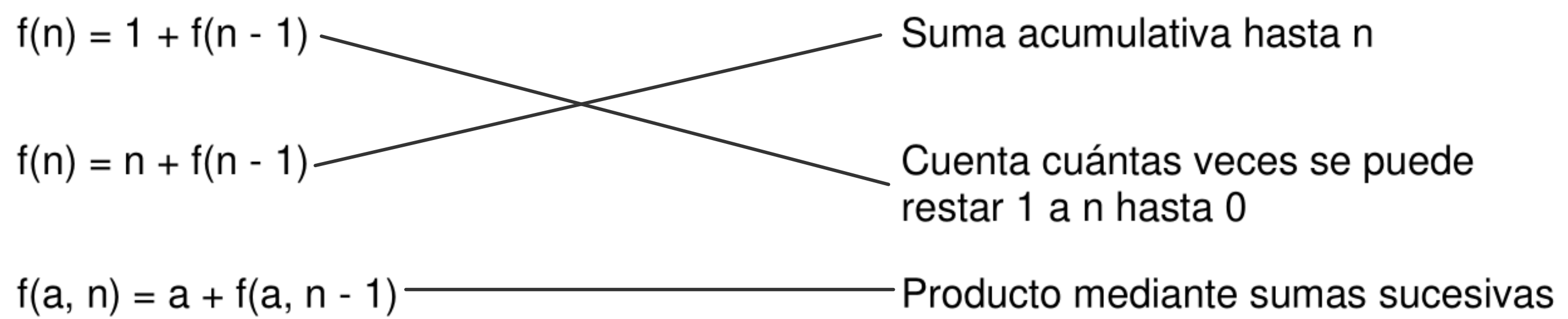
```
1. int potencia (int n, int b) {  
    if (b == 0) return 1;  
    return n * potencia (b - 1);  
}
```

2. Falta el caso base o condición de corte en la función recursiva lo que hace que nunca termine.

```
3. int cantidadDigitos (int numero) {  
    if (numero < 10) return 1  
    return 1 + cantidadDigitos (numero / 10);  
}
```


Unir frases

1. Relacione la función con su comportamiento considerando la condición de corte de todas que al ser n igual a 0 retornarán 0:



Verdadero o Falso

1. Una función recursiva siempre necesita al menos un caso base.
2. Una función con múltiples llamadas recursivas es más eficiente que una simple.